Estrutura de Dados II

Prof. Vanessa Matias Leite

1

Tabela de Espalhamento

(Hash/Tabela de disperção)

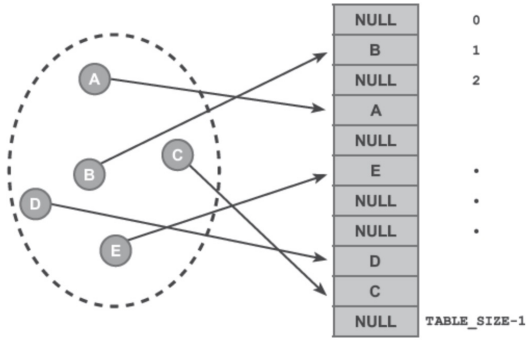
2

Tabela Hash

- É uma generalização da ideia de array;
- Sua ideia central é utilizar uma função, chamada função de hashing, para espalhar os elementos que queremos armazenar na tabela.
- Esse espalhamento faz com que os elementos fiquem dispersos de forma não ordenada dentro do array que define a tabela.

3

Tabela Hash



4

Tabela Hash

- Seu objetivo é a partir de uma chave simples, fazer uma busca rápida e obter o valor desejado.
- A Tabela Hash leva em conta o valor absoluto de cada chave, interpretado como um valor numérico.
- Através da aplicação de uma função conveniente, a chave é transformada em um endereço de uma tabela.

5

Tabela Hash

Por que espalhar os elementos melhora a busca?

- A tabela permite a associar valores a chaves
  - chave: parte da informação que compõe o elemento a ser inserido ou buscado na tabela
  - valor: é a posição (índice) onde o elemento se encontra no array que define a tabela
- Assim, a partir de uma chave podemos acessar de forma rápida uma determinada posição do array
  - Na média, essa operação tem custo O(1)

6

Tabela Hash

Vantagens:

- Alta eficiência na operação de busca ;
  - Caso médio é O(1) enquanto o da busca linear é O(N) e a da busca binária é $O(\log_2 N)$
- Tempo de busca é praticamente independente do número de chaves armazenadas na tabela;
- Implementação simples;

7

Tabela Hash

Desvantagens:

- Alto custo para recuperar os elementos da tabela ordenados pela chave.
  - Nesse caso, é preciso ordenar a tabela.
- O pior caso é O(N), sendo N o tamanho da tabela
- Alto número de colisões

8

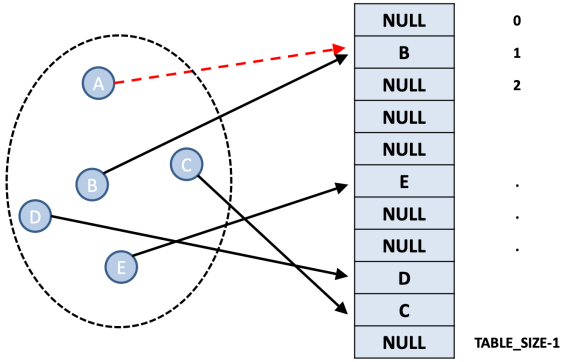
Tabela Hash

O que é uma colisão???

- Uma colisão ocorre quando duas (ou mais) chaves diferentes tentam ocupar a mesma posição na tabela hash.
- A colisão de chaves não é algo exatamente ruim, é apenas algo indesejável, pois diminui o desempenho do sistema.

9

Tabela Hash – Exemplo de Colisão



10

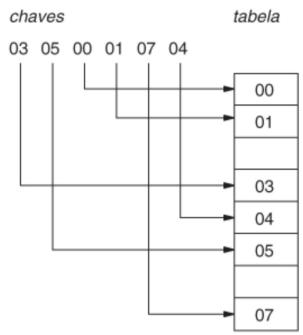
Princípios de Funcionamento – Acesso direto

- Suponha que existam n chaves a serem armazenadas em uma tabela T, sequencial e de tamanho m.
- As posições da tabela se situam no intervalo [0, m-1].
- Isto é, a tabela é particionada em m compartimentos, cada uma corresponde a um endereço e podendo armazenar r nós distintos.

11

Princípios de Funcionamento – Acesso direto

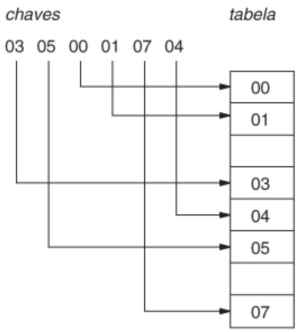
- O objetivo é armazenar cada chave no bloco referente ao seu endereço.
- A busca, assim, requer somente um acesso a um bloco.



12

Princípios de Funcionamento – Acesso direto

- Valor da chave como seu índice na tabela.
- Cada chave x é adicionada no compartimento x.



13

Princípios de Funcionamento – Acesso direto

- Chaves nem sempre são valores numéricos.
- As chaves podem consistir em nomes de pessoas.

Solução: Todo dado não numérico corresponde uma representação numérica no computador.
Assim sendo, todas as chaves são consideradas numéricas.

14

Princípios de Funcionamento – Acesso direto

- Vamos utilizar sempre o aceso direto?

  NÃO

- A quantidade de espaços vazios pode ser proibitiva.
- Como exemplo extremo, seja um conjunto de duas chaves apenas, de valores 0 e 999.999, respectivamente. A aplicação da técnica de acesso direto conduziria a uma tabela com 1.000.000 compartimentos, dos quais apenas dois ocupados.

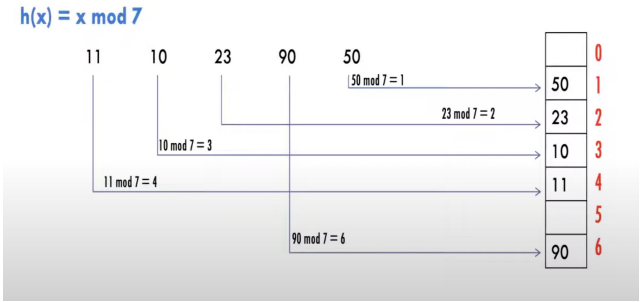
15

Princípios de Funcionamento

- Transformar cada chave x em um valor no intervalo [0, m-1].
- Através de uma função de dispersão h.
- Dada a chave x, determina-se o valor h(x), denominado endereço-base de c.
- Se o compartimento h(x) estiver desocupado, poderá ser utilizado para armazenar a x.

16

Princípios de Funcionamento



17

Princípios de Funcionamento

- A função de dispersão pode não garantir a injetividade.
- É possível a existência de outra chave y ≠ x, tal que h(y) = h(x).
- Esse fenômeno é denominado de colisão.
- As chaves x e y são sinônimas em relação a h.
- Na ocorrência desse fato, utiliza-se um procedimento especial, denominado tratamento de colisão.

18

Princípios de Funcionamento

- Uma função de dispersão h transforma uma chave x em um endereço-base h(x) da tabela de dispersão.
- Idealmente, uma função de dispersão deve satisfazer às seguintes condições:
  - produzir um número baixo de colisão;
  - ser facilmente computável;
  - ser uniforme (mesma probabilidade de serem escolhidos).

19

Funções Hash

- Método da divisão;
- Método da dobra;
- Método da multiplicação;
- Método da Análise dos Dígitos;

20

Funções Hash – Método da divisão

- Fácil e eficiente, por isso muito empregado.
- A chave x é dividida pela dimensão da tabela m, e o resto da divisão é usado como endereço-base. Isto é,

$$h(x) = h \bmod m,$$

resultando em endereços no intervalo [0, m – 1].

21

Funções Hash – Método da divisão

- Alguns valores de m são melhores que outros.
- Se m é um número par, h(x) será par quando x for par e ímpar quando x for ímpar. Não é uma boa solução.
- Se m for uma potência de 2, h(x) dependerá apenas de alguns dígitos de x. A situação é pior.

22

Funções Hash – Método da divisão

- Existem alguns critérios que têm sido aplicados com bons resultados práticos.
- Escolher m de modo que seja um número primo não próximo a uma potência de 2.
- Escolher m tal que não possua divisores primos menores que 20.

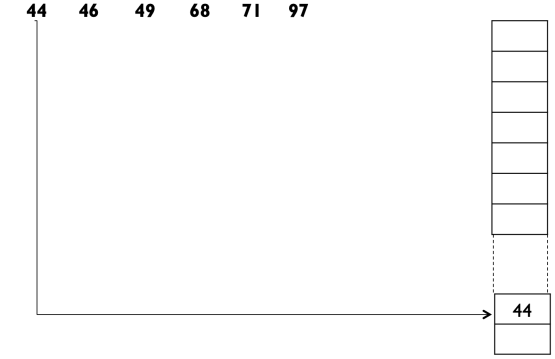
23

Funções Hash – Método da divisão

m = 23

44 mod 23 = 21

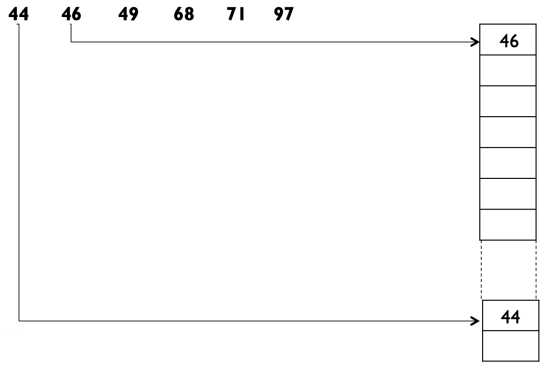
44 46 49 68 71 97



24

Funções Hash – Método da divisão

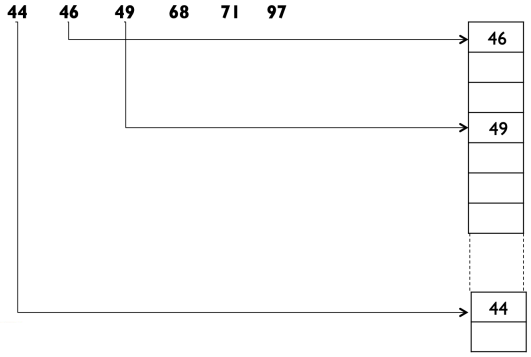
$m = 23$ 46 mod 23 = 0



25

Funções Hash – Método da divisão

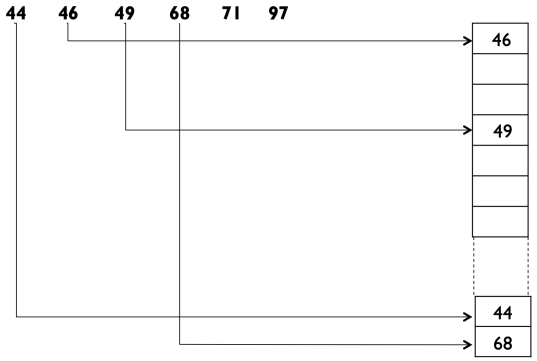
$m = 23$ 49 mod 23 = 3



26

Funções Hash – Método da divisão

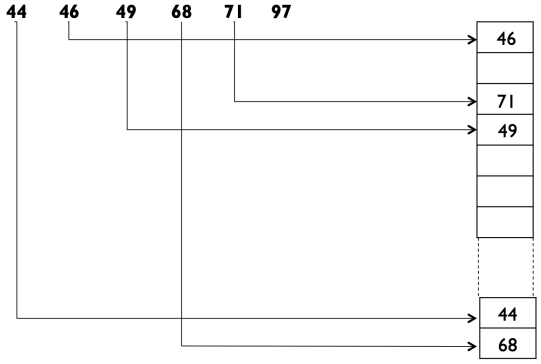
$m = 23$ 68 mod 23 = 22



27

Funções Hash – Método da divisão

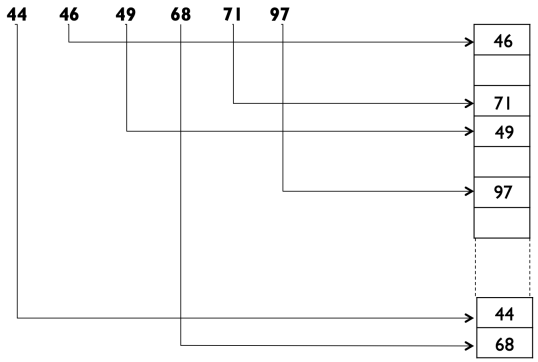
$m = 23$ 71 mod 23 = 2



28

Funções Hash – Método da divisão

$m = 23$ 97 mod 23 = 5



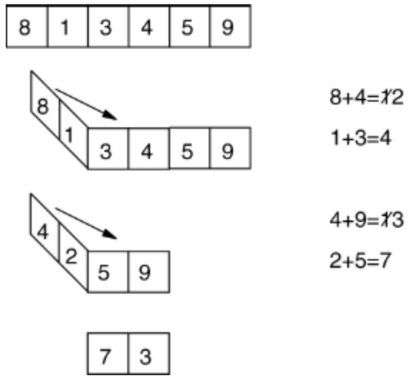
29

Funções Hash – Método dobra

- Suponha a chave como uma sequência de dígitos escritos num pedaço de papel.
- O método em questão consiste em “dobrar” esse papel, de maneira que os dígitos se sobreponham.
- Esses devem ser somados sem levar em consideração o “vai um”.

30

Funções Hash – Método dobra



8+4=12

1+3=4

4+9=13

2+5=7

31

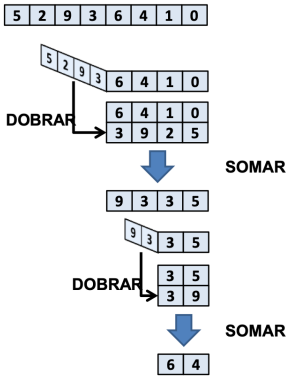
Funções Hash – Método dobra

- Chave: 279384
- t=2
- d=6

- Dobra 27 em 93 e soma, descartando “vai um” = 65.
- Resultado Intermediário: 6584
- Dobra 65 em 84 e soma ... Resultado final: 30

32

Funções Hash – Método dobra



33

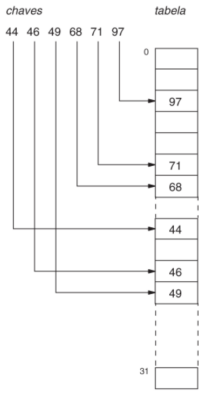
Funções Hash – Método dobra

- Pode ser utilizado com números binários
- Utilizando bits
- OU Exclusivo
- Chave: 71
- 71 = 00010 00111

```
 00010
 00111   ou exclusivo
 -----
 00101   = 5
```

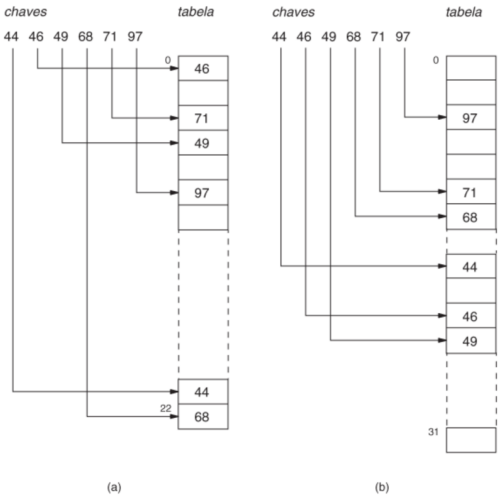
34

Funções Hash – Método dobra



71 = 00010 00111 endereço-base: (00010) **ouex** (00111) = 00101 = 5
46 = 00001 01110 endereço-base: (00001) **ouex** (01110) = 01111 = 15

35



36

Funções Hash – Método da multiplicação

- Apresenta algumas variações, sendo a mais conhecida o método do “meio do quadrado”.
- A chave é multiplicada por ela mesma.
- O resultado é armazenada em palavra de memória de b bits.
- O número de bits necessário para forma o endereço-base de uma chave é então retirado dos b bits.
- Descartando-se os bits excessivos da extrema direita e da extrema esquerda.

37

Funções Hash – Método da multiplicação

- Exemplo: chave 12
- 12 x 12 = 144
- 144 representado em binário: 10010000
- Armazenar em 10 bits: 0010010000
- Obter endereço de 6 bits (endereços entre 0 e 63)

0	0	1	0	0	1	0	0	0	0

38

Funções Hash – Método da multiplicação

- Exemplo: chave 12
- 12 x 12 = 144
- 144 representado em binário: 10010000
- Armazenar em 10 bits: 0010010000
- Obter endereço de 6 bits (endereços entre 0 e 63)

39

Funções Hash – Método da multiplicação

- Exemplo: chave 12
- 12 x 12 = 144
- 144 representado em binário: 10010000
- Armazenar em 10 bits: 0010010000
- Obter endereço de 6 bits (endereços entre 0 e 63)

40

Funções Hash – Método da análise de dígito

- Leva em consideração o conhecimento prévio do tipo de chave que se busca.
- A função de dispersão consiste em selecionar, de forma conveniente, alguns dos dígitos decimais que formam a chave para compor o seu endereço-base.

41

Funções Hash – Método da análise de dígito

Chaves: 443 462 492 681 712 972

- 1° Digito:{4 4 4 6 7 9 } -> 3 quatros, 1 seis, 1 sete e 1 nove.
- 2° Digito: { 4 6 9 8 1 7 } -> 1 um, 1 quatro, ...
- 3° Digitos: { 3 2 2 1 2 2 } -> 1 três, 4 dois, 1 um.
- Escolhendo o segundo dígito como endereço base não teríamos nenhuma colisão em uma tabela com 10 compartimentos .

42

Hashing perfeito

- Nunca ocorre colisão
- Chaves diferentes irão sempre produzir posições diferentes
- No pior caso, as operações de busca e inserção são sempre executadas em tempo constante, O(1).
- É utilizado onde a colisão não é tolerável
- Trata-se de um tipo de aplicação muito especifica, por exemplo, o conjunto de palavras reservadas de uma linguagem de programação. Nesse caso, conhecemos previamente o conteúdo a ser armazenado na tabela

43

Hashing imperfeito

- Para duas chaves diferentes a saída da função de hashing é a mesma posição na tabela. Ou seja, podem ocorrer colisões das chaves
- A colisão de chaves não é algo exatamente ruim, é apenas algo indesejável, pois diminui o desempenho do sistema
- De modo geral, muitas tabelas hash fazem uso de alguma outra estrutura de dados para lidar com o problema da colisão.

44

Tratamento de Colisões

- Mundo ideal
  - Hashing perfeito
  - Função de hashing irá sempre fornecer posições diferentes para cada uma das chaves inseridas;
- Mundo real
  - Independente da função de hashing utilizada, a mesma vai retornar a mesma posição para duas chaves diferentes: colisão!

45

Tratamento de Colisões

- Uma escolha adequada do tamanho da tabela pode minimizar as colisões.
  - Colisões ocorrerem porque temos mais chaves para armazenar do que o tamanho da tabela suporta;
  - Não há espaço suficiente para todas as chaves;

46

Tratamento de Colisões

- Uma escolha adequada da função de hashing pode minimizar as colisões
- Escolher uma função que produza um espalhamento uniforme das chaves reduz o número de colisões
- Infelizmente, não se pode garantir que as funções de hashing possuam um bom potencial de espalhamento por que as colisões também são uniformemente distribuídas.
- Colisões são teoricamente inevitáveis. Por isso, devemos sempre ter uma abordagem para tratá-las.

47

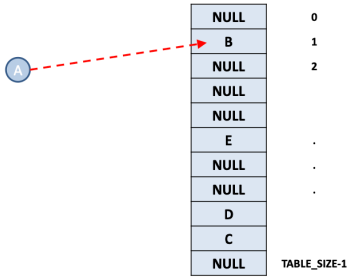
Tratamento de Colisões - Endereçamento aberto

- Também conhecido como open *addressing ou rehash*
- No caso de um colisão, percorrer a tabela hash buscando por uma posição ainda não ocupada
- Os elementos são armazenados na própria tabela hash
- Evita o uso de listas encadeadas

48

Tratamento de Colisões - Endereçamento aberto

A tenta ocupar a posição de B



49

Tratamento de Colisões - Endereçamento aberto

Vantagens:

- Maior número de posições na tabela para a mesma quantidade de memória usada no encadeamento separado;
- A memória utilizada para armazenar os ponteiros da lista encadeada no encadeamento separado pode ser aqui usada para aumentar o tamanho da tabela, diminuindo o número de colisões;

50

Tratamento de Colisões - Endereçamento aberto

Vantagens:

- Busca é realizada dentro da própria tabela. Recuperação mais rápida de elementos;
- Voltada para aplicações com restrições de memória;
- Ao invés de acessarmos ponteiros extras, calculamos a sequência de posições a serem armazenadas.

51

Tratamento de Colisões - Endereçamento aberto

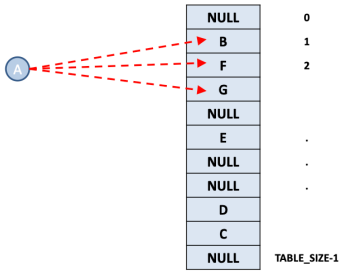
Desvantagens:

- Maior esforço de processamento no cálculo das posições;
- Esse esforço maior se deve ao fato de que, quando uma colisão ocorre, devemos calcular uma nova posição da tabela;
- Colisões sucessivas

52

Tratamento de Colisões - Endereçamento aberto

- Colisões sucessivas



53

Tratamento de Colisões - Endereçamento aberto

Para a realização do cálculo da nova posição após a colisão, existem três estratégias:

- Sondagem linear
- Sondagem quadrática
- Duplo hash

54

Tratamento de Colisões - Endereçamento aberto

Sondagem linear:

- Também conhecida como tentativa linear, espalhamento linear ou rehash linear
- Tenta espalhar os elementos de forma sequencial a partir da posição calculada utilizando a função de hashing

55

Tratamento de Colisões - Endereçamento aberto

Sondagem linear:

NULL	0
NULL	1
NULL	2
NULL	3
NULL	4
NULL	5
NULL	6
NULL	7
NULL	8
NULL	9
NULL	10

CHAVE	POSIÇÃO	INSERÇÃO
A	2	Posição 2 vazia. Insere elemento
B	6	Posição 6 vazia. Insere elemento
C	2	Posição 2 ocupada, procura na próxima posição: 3 Posição 3 vazia. Insere elemento
D	10	Posição 10 vazia. Insere elemento
E	10	Posição 10 ocupada, procura na próxima posição. Como a posição 10 é a última, volta para o início: 0 Posição 0 vazia. Insere elemento

E	0
NULL	1
A	2
C	3
NULL	4
NULL	5
B	6
NULL	7
NULL	8
NULL	9
D	10

56

Tratamento de Colisões - Endereçamento aberto

Sondagem linear:

- Apresenta um problema conhecido como agrupamento primário
- A medida que a tabela hash fica cheia, o tempo para incluir ou buscar um elemento aumenta
- A medida que os elementos são inseridos surgem longas sequências de posições ocupadas
- A ocorrência desses agrupamentos aumenta o tempo de pesquisa, diminuindo o desempenho

57

Tratamento de Colisões - Endereçamento aberto

Sondagem quadrática:

- Tenta espalhar os elementos utilizando uma equação do segundo grau;

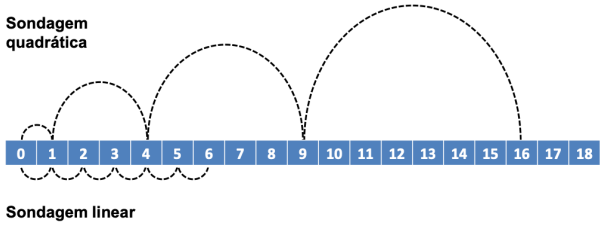
pos + (c1 * i) + (c2 * i2)

- pos é a posição obtida pela função de hashing
- i é tentativa atual
- c1 e c2 são os coeficientes da equação

58

Tratamento de Colisões - Endereçamento aberto

Sondagem quadrática



59

Tratamento de Colisões - Endereçamento aberto

Sondagem quadrática

- Resolve o problema de agrupamento primário
- Porém, gera outro problema conhecido como agrupamento secundário
- Todas as chaves que produzam a mesma posição inicial também produzem as mesmas posições na sondagem quadrática
- Felizmente, a degradação produzida pelos agrupamentos secundários ainda é menor que a produzida pelos agrupamentos primários

60

Tratamento de Colisões - Endereçamento aberto

Duplo Hash:

- Tenta espalhar os elementos utilizando duas funções de hashing:
- a primeira função de hashing, H1, é utilizada para calcular a posição inicial do elemento
- a segunda função de hashing, H2, é utilizada para calcular os deslocamentos em relação a posição inicial (no caso de uma colisão)

61

Tratamento de Colisões - Endereçamento aberto

Duplo Hash:

- A posição de um novo elemento na tabela hash é obtida como sendo:

$$H1 + i * H2$$

onde i é tentativa atual de inserção do elemento

- É necessário que as duas funções de hashing sejam diferentes.
- A segunda função de hashing não pode resultar em um valor igual a ZERO pois, neste caso, não haveria deslocamento

62

Inserção com tratamento de colisão

- Calcular a posição da chave no array
- Recalcular a posição enquanto houver colisão (limitar o número de tentativas)
- Alocar espaço para os dados
- Armazenar os dados na posição calculada

63

Busca com tratamento de colisão

- Calcular a posição da chave no array
- Verificar se há dados na posição calculada e se esses dados combinam com a chave
- Recalcular a posição enquanto os dados forem diferentes da chave
- Retornar os dados

64

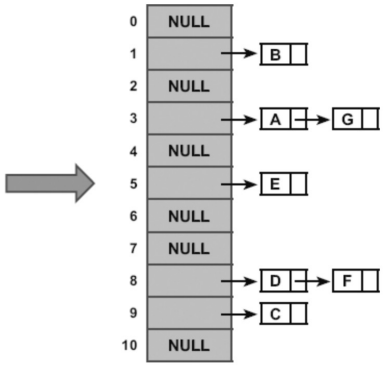
Encadeamento Separado

- Não procura por posições vagas (valor NULL) dentro do array que define a tabela
- Armazena dentro de cada posição do array o início de uma lista dinâmica encadeada
- É dentro dessa lista que serão armazenadas as colisões (elementos com chaves iguais) para aquela posição do array

65

Encadeamento Separado

CHAVE	POSIÇÃO
A	3
B	1
C	9
D	8
E	5
F	8
G	3



66

Encadeamento Separado

- A lista dinâmica encadeada mantida em cada posição da tabela pode ser ordenada ou não
- Lista não ordenada:
  - Inserção tem complexidade O(1) no pior caso: basta inserir o elemento no início da lista.
  - Busca tem complexidade O(M) no pior caso: busca linear
- Desvantagem
  - Quantidade de memória consumida: gastamos mais memória para manter os ponteiros que ligam os diferentes elementos dentro de cada lista

67

Aplicações de Hash

- A tabela hash pode ser utilizada para busca de elementos em base de dados;
  - estruturas de dados em memória, bancos de dados e mecanismos de busca na Internet;
- verificação de integridade de dados e autenticação de mensagens
  - os dados são enviados juntamente com o resultado da função de hashing
  - Quem receber os dados recalcula a função de hashing usando os dados recebidos e compara o resultado obtido com o que ele recebeu.
  - Resultados diferentes: erro de transmissão

68

Aplicações de Hash

- Armazenamento de senhas com segurança
  - a senha não é armazenada no servidor, mas sim o resultado da função de hashing
- Implementação da tabela de símbolos dos compiladores;
- Criptografia
  - MD5 e família SHA (Secure Hash Algorithm).

69